Embedded Lab

Embedded Systems tutorials, projects, and more ...



Check out some cool electronic projects here!
electronics-lab.com

LAB 20: INTERFACING A KS0108 BASED GRAPHICS LCD (PART 1)

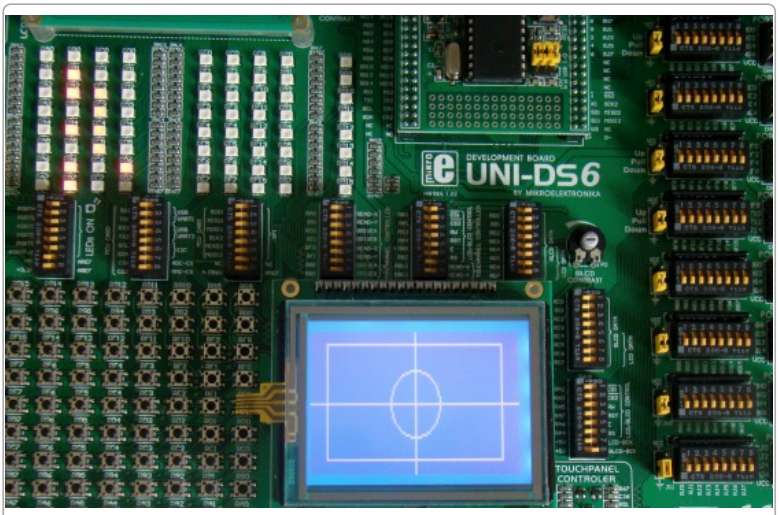
Posted on December 20, 2011 | by R-B | 29 comments |

Tesla Generator? $49
Eliminate Your Power Bill Easy Do It Yourself. Great Discount Now!

The use of a graphical LCD (GLCD) drastically changes the look of your project. It provides more freedom for presenting data than the HD44870 based character LCDs. Today we will see how to interface a KS0108 (name of the display controller chip) based GLCD to a PIC microcontroller. This experimental tutorial is divided into two parts. In the first part, we will see how to write a firmware for the PIC microcontroller to initialize the GLCD and send data to plot points and lines on the screen. The second part will focus more on exploring the built-in GLCD Library of mikroC Pro for PIC compiler to display more complex texts and objects. Since GLCDs are real resource hungry devices (in terms of required I/O pins and memory), a bigger size PIC microcontroller (PIC16F887, which has 36 I/O pins and 14KB flash memory) is selected for this experiment. I am using MikroElektronika's UNI-DS6 development board to demonstrate this project, but the circuit setup can also be made on a breadboard.

Search...

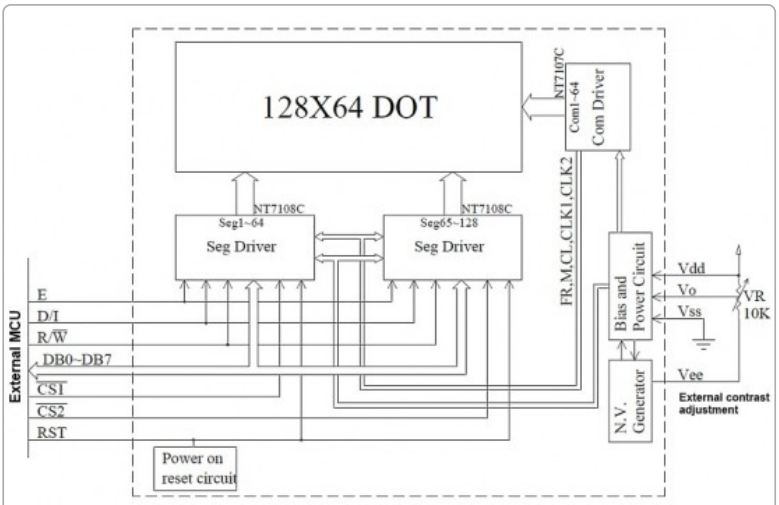

WORLDWIDE SHIPPING



Interfacing a 128x64 pixels GLCD

Theory

The graphical LCD used in this experiment is Winstar's WDG0151-TMI module, which is a 128×64 pixel monochromatic display. It uses two Neotic display controller chips: NT7108C and NT7107C, which are compatible with Samsung KS0108B and KS0107B controllers. The KS0108B (or NT7108C) is a dot matrix LCD segment driver with 64 channel output, and therefore, the WDG0151 module contains two sets of it to drive 128 segments. On the other hand, the KS0107B (or NT7107C) is a 64-channel common driver which generates the timing signal to control the two KS0108B segment drivers. The KS0108B and KS0107B are a very popular controllers and have made their way into many graphical LCDs. The internal block diagram of the WDG0151 GLCD module is shown below.



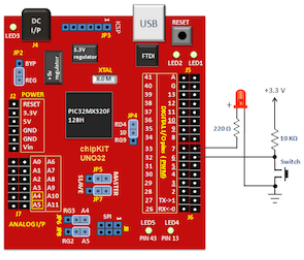
Internal block diagram of a KS0108B (NT7108C) based 128x64 pixel GLCD module

The NT1707C drives the 64 display lines, COM1 – COM64. The first NT7108C drives the left half segments (SEG1 to SEG64) and the second one drives the right half segments (SEG65 to SEG128) of the display. The two halves of the display can be individually accessed through the chip select pins (CS1 and CS2) of the two NT7108C drivers. Each half

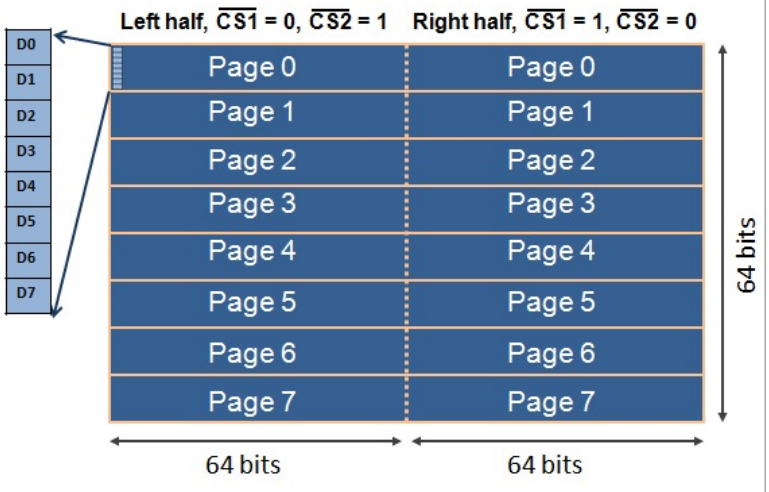


CHIPKIT TUTORIALS AND PROJECTS

chipKIT is an open source embedded development environment that uses powerful 32-bit PIC processors for hardware. Check out these free comprehensive tutorials and DIY projects that are intended to facilitate the learning of chipKIT interfacing and programming.



consists of 8 horizontal pages (0-7) which are 8 bits (1 byte) high. This is illustrated in the drawing below.



GLCD pages

Starting from page 0 on the left half (/CS1 = 0) if you transmit one data byte, it will appear on the first column of page 0. If you repeat this 64 times, then switch to the second half, and repeat until 128th position is reached, the first 8 display lines will be plotted. The next 8 lines can be plotted similarly by switching to page address 1. The total amount of bytes needed for a complete display frame (128×64 pixels) is, therefore, 2 * 64 pixels * 8 bits = 1024 bytes.

The Winstar WDG0151-TMI module does have an internal negative voltage generator circuit which provides a negative voltage at V_{EE} external pin. An external potentiometer (usually 10 K) is connected between Vcc and V_{EE} pins to set the LCD working voltage (contrast) at Vo pin. The pin diagrams of KS0108 based GLCDs is not standardized and it is therefore, important to read the manufacturer's datasheet for correct wiring of a GLCD module. The following table shows the pin descriptions of Winstar WDG0151-TMI module. It has altogether 20 pins. The first two pins (1 and 2) are the chip select pins for the left and right display controller. They are active low in a WDG0151-TMI module, but they could be active high in some other models. That's why I said reading manufacturer's datasheet is very important. The WDG0151-TMI module operates at 5.0 V power supply. Pin number 6 is Data/Instruction (also called Register Select, RS) select pin. The 8-bit data fed to D0-D7 pins of the GLCD is received by the LCD controller chip as an instruction if D/I = 0, and as data if D/I is 1. The R/W and E pins have similar functions as in a HD44780 based character LCD module. A fixed resistor value must be connected in series with the back-light LED (pins 19 and 20) to limit the current.

Share

AdChoices
► LCD Display
► Graphic LCD
► LCD 128X64

SUBSCRIBE

ABOUT ME



Raj Bhatt Electronics enthusiast and founder of Embedded Lab

Read More

G+1 0 Follow @EmbeddedLab

Subscribe through email

Sign Up

I SELL ON TINDIE

Read Our Privacy Policy!

Embedded Lab
16,586 likes

Like Page

AdChoices
► Arduino LCD
► Graphic Display
► Embedded LCD

Pin no.	Symbol	Level	Description
1	$\overline{CS1}$	L	Select segments 1-64
2	$\overline{CS2}$	L	Select segments 65-128
3	V_{SS}	0 V	Ground
4	V_{DD}	5.0 V	Supply voltage for logic
5	Vo	Variable	Contrast adjustment
6	D/I or RS	H/L	H : Data, L: Instruction
7	R/W	H/L	H: Read data, L: Write data
8	E	H	Enable signal
9-16	D0-D7	H/L	Data bus
17	RST	L	Reset the LCD module
18	V_{EE}		Negative voltage output
19	A		Anode (+) of B/L LED
20	K		Cathode (-) of B/L LED

Pin description of Winstar WDG01510 GLCD module

The KS0107/KS0108 does not have a character generator so this must be implemented in the microcontroller firmware. The LCD controller supports a handful of instructions which are summarized in the table shown below. Note that the RS (D/I) pin is high only during data read and data write operations, and stays low when a transmitted byte is an instruction.

Instruction	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Function
Display on/off	L	L	L	L	H	H	H	H	H	L/H	Controls the display on or off. Internal status and display RAM data is not affected. L:OFF, H:ON
Set address (Y address)	L	L	L	H	Y address (0-63)						Sets the Y address in the Y address counter.
Set page (X address)	L	L	H	L	H	H	H	Page (0-7)			Sets the X address at the X address register.
Display Start line (Z address)	L	L	H	H	Display start line (0-63)						Indicates the display data RAM displayed at the top of the screen.
Status read	L	H	Busy	L	On/Off	Reset	L	L	L	L	Read status. BUSY L: Ready H: In operation ON/OFF L: Display ON H: Display OFF RESET L: Normal H: Reset
Write display data	H	L	Write data								Writes data (DB0: 7) into display data RAM. After writing instruction, Y address is increased by 1 automatically.
Read display data	H	H	Read data								Reads data (DB0: 7) from display data RAM to the data bus.

Display control instructions (click on image to enlarge)

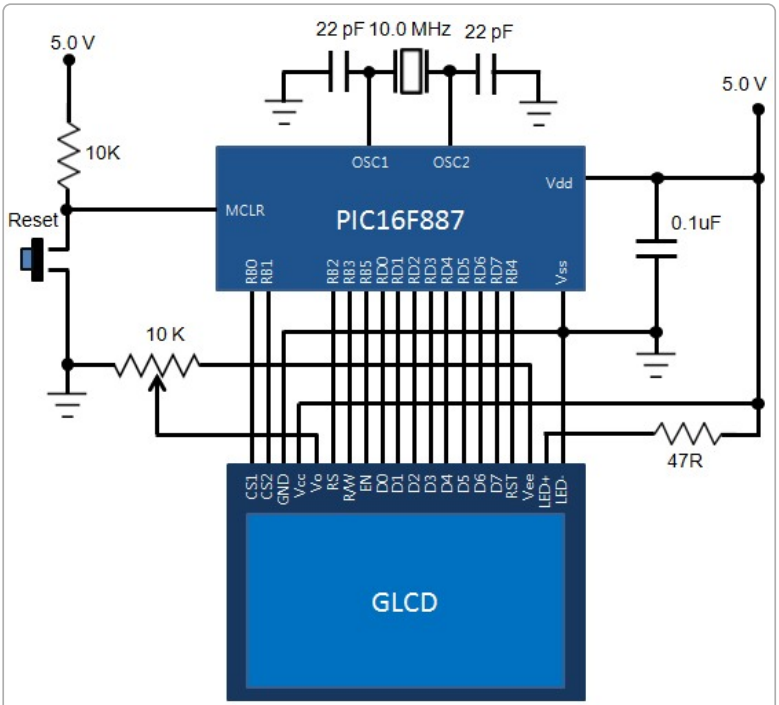
Circuit diagram

The circuit diagram for this experiment is shown below. The PIC16F887 microcontroller is used to drive a Winstar WDG0151-TMI GLCD. The data pins are connected to PORTD and other control signals are driven through PORTB pins. I am trying this circuit on MikroElektronika's UNI-DS6 development board.

Share

CATEGORIES

- 555 Timer
- Analog
- Arduino
- ARM projects
- AVR Projects
- AVR Tutorials
- BeagleBone
- chipKIT
- Display
- dsPIC
- Electric Imp
- Embedded Lab Projects
- Embedded Labs
- Embedded Lessons
- ESP8266
- FPGA
- Hack
- Intel Edison
- Internet of Things
- LED display
- littleBits
- MCU develeopment tools
- Microcontroller Programmers
- MSP430 Launchpad
- Netduino
- Open-source
- PIC Projects
- PIC Tutorials
- PIC18F
- PIC32
- Power Supply
- Processing
- Product Review
- Products
- Raspberry Pi
- Robotics
- Sensors
- STM32
- Tech News
- Texas Instruments
- TI
- Tips and Tricks
- Tiva C
- Uncategorized
- wireless
- XMega



Circuit diagram for GLCD interfacing

AdChoices
► Excel Tutorial
► Pic LCD
► Arduino Projects

Software

We will be writing our test program in C using MikroElektronika's mikroC Pro for PIC compiler. Although, the compiler does provide built-in library routines for GLCD operations, we will first try to write our own test code for transferring display data from the PIC16F887 to the GLCD. Later, we will explore the MikroElektronika's GLCD library for more complex operations. The code provided below generates 11 dotted horizontal lines on the GLCD screen with a six-line spacing between two. I took most portion of it from Osama's Lab GLCD library and modified it to suit with mikroC Pro for PIC and WDG0151-TMI GLCD. Here's a brief description of various user-defined function subroutines used in the code.

Share

GLCD_ON() : This function turns the display on. This can be done by sending the command 3Fh to both the controllers. So, while sending this command, both CS1 and CS2 must be pulled low. Similarly the RS pin should be low too as the byte sent is an instruction.

Set_Start_Line() : This function changes the line number to be displayed at the top of the screen. You can set it to be any number between 0 to 63. It does not affect the data in the display RAM, it just scrolls the display up and down.

GOTO_COL() : Moves the cursor to specified column (0-127).

GOTO_ROW() : Moves the cursor to specified row or page number (0-7).

GOTO_XY() : Moves the cursor to specified row and column.

GLCD_Write() : Writes a byte of data to the current location.

GLCD_Read() : Returns a byte read from the current display location. If you see the code for this subroutine, you will see there are two read operations involved. The first one is a dummy read during which the data is fetched from the display RAM is latched in to the output register of KS0108B. In the second read, the microcontroller can get the actual data.

GLCD_ClrIn() : Clears a specified row (0-7).

GLCD_CLR() : Clears the whole screen (all 8 pages).

Draw_Point() : Plots a dark or light color point at a specified position.

At the end, the dotted lines are created by plotting too many points in horizontal directions.

```
/*
 * Project name: Testing GLCD with PIC16F887
 * Copyright:
     (c) Embedded-Lab.com, 2011.

 * Description:
     This routine demonstrates how to initialize a KS0108 based
     activate the pixels on display. A sub-routine is written to
     the GLCD at a given coordinates.
      * Test configuration:
     MCU:             PIC16F887
     Dev.Board:       UNI-DS6
     Oscillator:      HS, 10.0000 MHz
     Ext. Modules:    GLCD 128x64, KS108/107 controller

*/

// Glcd module connections
#define GLCD_Data   PORTD
#define GLCD_Dir    TRISD

sbit GLCD_CS1 at RB0_bit;
sbit GLCD_CS2 at RB1_bit;
sbit GLCD_RS  at RB2_bit;
sbit GLCD_RW  at RB3_bit;
sbit GLCD_RST  at RB4_bit;
sbit GLCD_EN at RB5_bit;

sbit GLCD_CS1_Direction at TRISB0_bit;
sbit GLCD_CS2_Direction at TRISB1_bit;
sbit GLCD_RS_Direction  at TRISB2_bit;
sbit GLCD_RW_Direction  at TRISB3_bit;
sbit GLCD_RST_Direction  at TRISB4_bit;
sbit GLCD_EN_Direction at TRISB5_bit;
// End Glcd module connections

void Enable_Pulse()
{
   GLCD_EN = 1; //EN high
   delay_us(5);
   GLCD_EN = 0; //EN low
   delay_us(5);
}

void GLCD_ON()
{
    //Activate both chips
    GLCD_CS1 = 0;
    GLCD_CS2 = 0;
    GLCD_RS = 0;          //RS low --> command
    GLCD_RW = 0;          //RW low --> write
    GLCD_Data = 0x3F;         //ON command
    Enable_Pulse();
```

Share

```
}

void Set_Start_Line(unsigned short line)
{
    GLCD_RS = 0;             //RS low --> command
    GLCD_RW = 0;             //RW low --> write
    //Activate both chips
    GLCD_CS1 = 0;
    GLCD_CS2 = 0;
    GLCD_Data = 0xC0 | line;     //Set Start Line command
    Enable_Pulse();
}

void GOTO_COL(unsigned int x)
{
   unsigned short Col_Data;
   GLCD_RS = 0;              //RS low --> command
   GLCD_RW = 0;              //RW low --> write
   if(x<64)             //left section
   {
      GLCD_CS1 = 0;          //select chip 1
      GLCD_CS2 = 1;          //deselect chip 2
      Col_Data = x;              //put column address on data po
   }
   else                 //right section
   {

      GLCD_CS2 = 0;
      GLCD_CS1 = 1;
      Col_Data = x-64;   //put column address on data port
   }
   Col_Data = (Col_Data | 0x40 ) & 0x7F;  //Command format
   GLCD_Data = Col_Data;
   Enable_Pulse();
}

void GOTO_ROW(unsigned int y)
{
   unsigned short Col_Data;
   GLCD_RS = 0;                  //RS low --> command
   GLCD_RW = 0;                  //RW low --> write
   Col_Data = (y | 0xB8 ) & 0xBF; //put row address on data port
   GLCD_Data = Col_Data;
   Enable_Pulse();
}

void GOTO_XY(unsigned int x,unsigned int y)
{
    GOTO_COL(x);
    GOTO_ROW(y);
}

void GLCD_Write(unsigned short b)
{
   GLCD_RS = 1;          //RS high --> data
   GLCD_RW = 0;          //RW low --> write
   GLCD_Data = b;            //put data on data port
   delay_us(1);
   Enable_Pulse();
}

unsigned short GLCD_Read(unsigned short column)
{
    unsigned short read_data = 0; //Read data here
    GLCD_Dir = 0xFF;     //PORTD as Input
    GLCD_RW = 1;              //Read
    GLCD_RS = 1;              //Data
    GLCD_CS1 = (column>63);
    GLCD_CS2 = !GLCD_CS1;         //Disable/Enable CS2
    delay_us(1);        //tasu
    GLCD_EN = 1;              //Latch RAM data into ouput registe
    delay_us(1);        //twl + tf
```

Share

```
    //Dummy read
    GLCD_EN = 0;             //Low Enable
    delay_us(5);
    GLCD_EN = 1;             //latch data from output register t
    delay_us(1);             //tr + td(twh)

    read_data = GLCD_Data;    //Input data
    GLCD_EN = 0;             //Low Enable to remove data from th
    delay_us(1);        //tdhr
    GLCD_Dir = 0x00;     //Output again
    return read_data;
}

 void GLCD_Clrln(unsigned short ln)
 {
    int i;
    GOTO_XY(0,ln);      //At start of line of left side
    GOTO_XY(64,ln);     //At start of line of right side (Proble
    GLCD_CS1 = 0;
    for(i=0;i<65;i++)
    GLCD_Write(0);
 }

//-- ------------------------

void GLCD_CLR()
{
   unsigned short m;
   for(m=0;m<8;m++){
    GLCD_Clrln(m);
   }
}

void Draw_Point(unsigned short x,unsigned short y, unsigned shor
{
    unsigned short Col_Data;;
    GOTO_XY(x,(y/8));
    switch (color)
    {
        case 0:         //Light spot
            Col_Data = ~(1<<(y%8)) & GLCD_Read(x);
        break;
        case 1:         //Dark spot
            Col_Data = (1<<(y%8)) | GLCD_Read(x);
        break;
    }
    GOTO_XY(x,(y/8));
    GLCD_Write(Col_Data);
}

void main() {
  unsigned short u, v;
  ANSEL  = 0;                  // Configure AN pins as digital
  ANSELH = 0;
  C1ON_bit = 0;                // Disable comparators
  C2ON_bit = 0;
  TRISD = 0x00;
  TRISB = 0x00;
  PORTB = 0x00;
  PORTD = 0x00;
  GLCD_CS1 = 1;                // De-Activate both chips
  GLCD_CS2 = 1;
  GLCD_RST = 1;
  GLCD_ON();
  GLCD_CLR();
  Set_Start_Line(0);
  do {

     for(u=0; u<64; u+=6)
      for (v=0; v<128; v+=2)
      Draw_Point(v, u, 1);
```

Share

```
        delay_ms(1000);
        GLCD_CLR();
        delay_ms(1000);

    } while(1);
}
```

Download the Source code and HEX file

When the microcontroller runs the program, you will see 11 dotted horizontal lines displayed on the screen as shown below.



Dotted horizontal lines

Summary

In this experimental tutorial, the internal block diagram of a KS0108 based 128×64 pixel GLCD, its pin configurations, and instruction set were discussed briefly. The GLCD operation was demonstrated with a Winstar WDG0151 GLCD module interfaced to the PIC16F887 microcontroller. A very basic firmware was written in mikroC Pro for PIC to demonstrate how to activate selected pixels on the screen to draw a line. In the second part of this tutorial (will be posted soon) we will learn to use the built-in GLCD library of mikroC Pro for PIC compiler to draw more complex objects and texts on the screen.

Second part of this tutorial is here:

Lab 20: Interfacing a KS0108 based Graphics LCD (Part 2)

Share

EasyEDA: Free Circuit Design Tool & Low Cost PCB Prototype

10 pcs 2 layers only $10, register to get $5 Cash Coupon.

www.easyeda.com

G+1 0

Like 2

Related Posts:

1. How to use mikroElektronika's GLCD bitmap editor tool to convert a BMP image in to a data array
2. Lab 20: Interfacing a KS0108 based Graphics LCD (Part 2)
3. Lab 14: Inter-Integrated Circuit (I2C) communication
4. In-Circuit Debugging of PIC microcontrollers

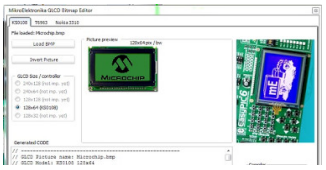


Powered by Google



High Voltage Resistors

Ad ohmcraft.com



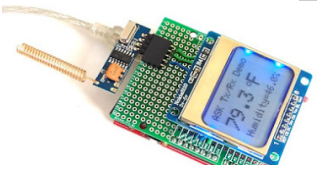
How to use mikroElektronika's GLCD bitmap edito...

embedded-lab.com



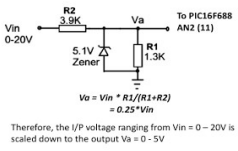
Tesla Generator? $49

Ad energyinnovator.org



Arduino Projects

embedded-lab.com



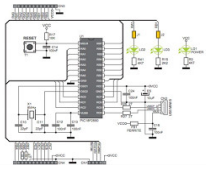
PIC-based Digital Voltmeter (DVM)

embedded-lab.com



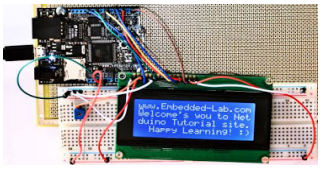
Free Arc Flash Handbook

Ad graphicproducts.com



Getting started with PIC18F Microcontrollers

embedded-lab.com



Netduino Tutorials and Projects

embedded-lab.com

Share

tagged with GLCD, GLCD library, KS0108, mikroC, PIC16F887, WDG0151, Winstar

PIC Tutorials PIC18F Tech News Tips and Tricks

29 COMMENTS

Abdullah Dokulmez February 25, 2016 4:44 am

Hi I have a problem with TG12864B-02WA0 display…. I want to use this LCD with ATMEGA16A-PU But when I programming this display don't working. I did't find any example project so can you share any library documantary for atmel

Reply

Ahmet February 5, 2016 11:09 am

Hi everybody;
In the description it says "An external potentiometer (usually 10 K) is connected between Vcc and VEE pins to set the LCD working voltage (contrast) at Vo pin"; however in the circuit diagram, the potentiometer had been connected between Gnd and VEE.
I am a little bit confused. Have I miss something?;
last night I destroyed one WG12864A because of lack of experience and my terrible mistake in Vee and Vo.

Thanx.

Reply

R-B February 5, 2016 11:23 am

Ahmet,
Since Vee is a negative voltage, connect the potentiometer between VCC and VEE to achieve a larger dynamic range for the contrast adjustment voltage.

Reply

abc July 3, 2015 9:01 am

kindly let me know which graphical lcd have you used i mean how can i choose it in proteus 8?

Reply

hamed June 16, 2015 5:29 am

Share

I am transfering this code for STM32 microcontroller,
in function: unsigned short GLCD_Read(unsigned short column)
there is : GLCD_CS1 = (column>63);
I dont know what is assigned to GLCD_CS1
would you please make this command clear?

Reply

jeevitha May 17, 2015 12:00 pm

I have a doubt that how to display an image in GLCD using ARM.I can get 25% of image as output.

Reply

James Goodwin May 16, 2015 4:06 pm

Could anyone help me interface an S6B0108 based 128×64 GLCD with an ArduinoMega2560?

Reply

Iqbal December 5, 2014 10:08 pm

plz help I am using glcd 164×28 of Topway LM12864DDY
http://www.topwaydisplay.com/Pub/Manual/LM12864DDY-Manual-Rev0.1.pdf
I am using CCS C Compiler & interfacing with pic16f877a
I have used many codes but could not get results data is not shown on LCD here is my code plz help

```
#include
#fuses HS,NOWDT,NOPROTECT,NOLVP
#use delay(clock=20000000)
#include "HDM64GS12.c"
#include
#include

void main() {
char voltText[] = "Volts";// warning[] = "Warning";
delay_ms(100);
glcd_init(ON);
glcd_fillScreen(OFF);
while(true)
{
glcd_rect(1, 5, 126, 15, NO, ON);
glcd_text57(70, 18, voltText, 1, ON);
glcd_circle(30, 47, 10, NO, ON);
}
}
```

Reply

Jon September 24, 2014 10:02 am

The diagram from Winstar says that the POT should be connected between VDD and VEE. You have it as VSS and VEE.

Share

Reply

mohamad July 24, 2014 3:38 am

hi
i want to know how can i do this project with Codevision software
it needs C language for programing
thanks

Reply

faari July 2, 2014 2:38 pm

I am writing a program which can take the analogue voltage reading and use adc to convert this signal into digital signal and display In terms of digital values. Initially I was using a digital lcd and displayed

the above mentioned data on lcd. Now I need a program which can display the data in the form of a dial (gauge) on GLCD with a needle showing the change in voltage.
I am using

1. PIC – C compiler
2. 18F452 micro controller
3. HDM64GS12 GLCD with a KS0108 display controller. The HDM64GS12 is 128 by 64 pixels GLCD
4. I have driver file “HDM64GS12.c”. This file contains drivers for using a Hantronix HDM64GS12 .
5. I also have a graphics file named ”graphics.c”. It contains functions to draw lines, rectangles, bars, circles and text to a display

Reply

Rocky May 25, 2014 10:06 pm

I want to know the sequence of instructions that needs to carried out to initialize Glcd

Reply

Phil Johns June 30, 2014 2:01 pm

it’s written above in that code. “GLCD_ON()”

Only thing that’s peculiar is… every glcd i’ve used…CS1 and 2 should be set during initializing…

I believe there’s errors in this code, as identified in other comments…clearline funciton needs to be modified to the code in the comment below

Reply

Cristhian Carrillo April 8, 2014 8:41 pm

Good afternoon 😀 I nedd your help. How I can plot an analog signal using the PIC16F887 and this GLCD? ¡Pleeaaase!

Share

Reply

mahshid November 22, 2013 11:01 am

thanks..

Reply

Nenad May 2, 2013 7:03 am

BTW, this function should look like this:

```
void GLCD_ClrIn(unsigned short In)
{
int i;
GOTO_XY(0,In);
GLCD_CS1 = 0;
GLCD_CS2 = 1;
```

```
for(i=0;i<65;i++)
GLCD_Write(0);

GOTO_XY(64,ln);
GLCD_CS1 = 1;
GLCD_CS2 = 0;
for (i=0; i<65; i++)
GLCD_Write(0);
}
```

Reply

Nenad May 2, 2013 6:50 am

Good tutorial! Thanks!

Reply

ENG.ASMA QASSEM January 19, 2013 5:42 am

THANK YOU ABOUT THIS IMPOTRANT INFORMATION >>>>
I WILL BE THANKFULL IF YOU SUPPLY ME WITH TUTORIAL ABOUT INTERFACE AVR ATMEL WITH 64128Q DISPLAYTECH GRAPHIC LCD....ESPECIALLY IF ITS BY ASSEMBLY LANGUAGE>>.

Reply

amir December 24, 2012 1:56 pm

you are best
thank you

Reply

dave April 2, 2012 4:38 pm

Never mind .. looks like I'd forgotten to GLCD_init for the mikroC libraries 🙂

Reply

Share

dave April 2, 2012 4:33 pm

I'm finding that the mikroC built in graphic libraries don't get along with the code here. I'm sure it's grabbing pointers to things that the mikroC libraries need .. any thoughts on how to get them to co-exist ? I need the speed of the "GLCD_Write" function and the versatility of the mikroC libraries both.

Thanks !

Reply

Pingback: tutorials for Graphic LCD

Pingback: Reviewing the iCA05 Graphic LCD development kit from iCircuit Technologies » Geko Geek

Pingback: Electronics-Lab.com Blog » Blog Archive » Interfacing a KS0108 based Graphics LCD (Part 2)

Pingback: Interfacing a KS0108 based Graphics LCD (Part 2) » Geko Geek

Alexander December 30, 2011 5:54 am

i think in microC there is a built in library for working with KS108 displays ?

Reply

R-B January 2, 2012 10:03 am

Alexander,
Yes it does.

Reply

Shahrul December 30, 2011 5:33 am

Hi, I'm trying to display this GLCD. I have copied the source code. The different is, I don't have 10K variable, so I use 1K variable. I measure voltage at Vee is -6V. Is that, because of this my GLCD dosn't display anythig?

Reply

R-B January 2, 2012 10:03 am

Shahrul,
Vee is negative because of the negative voltage generator built inside the module. It is normal. Try adjusting the LCD contrast by varying the potentiometer. Make sure you have connected everything correctly.

Reply

Share

LEAVE A REPLY

Your email address will not be published. Required fields are marked *

Name *

Email *

Website

Comment

Post Comment

© 2016 Embedded-Lab. All Rights Reserved.